FLOMERCE

Flomerce
Starter Learning Guide

& Employee Onboarding Handbook

Version 1.0 · April 2026

Table of contents

1. **Welcome to Flomerce** — what we are, who we serve, how to use this guide
2. **The big picture** — multi-tenant SaaS in plain English, with a request-flow diagram
3. **Vocabulary** — business terms and technical terms, defined with real examples
4. **Core concepts explained** — HTML/CSS/JS, React, APIs, databases, Cloudflare Workers
5. **Guided tour of the codebase** — what lives in each folder and where to peek first
6. **Trace your first feature: User Login** — every file involved, with code and a sequence diagram
7. **The debugging playbook** — DevTools, "what do I check first?" flowchart, common errors
8. **Third-party services cheat sheet** — Razorpay, Brevo, Shiprocket, Google, Push, WhatsApp
9. **90-day learning roadmap** — week-by-week study plan with free resources
10. **Quick-reference appendix** — important files, common commands, "where do I look if X is broken?"

1. Welcome to Flomerce

What Flomerce is

Flomerce is a **multi-tenant SaaS platform** that lets a small business owner sign up, create an online store at `their-store.flomerce.com`, list products, accept payments, and ship orders — all without writing code. Think of it as one big building where every shop gets its own clearly-labelled unit, but they all share the same plumbing, electricity, and front desk.

The platform is built on Cloudflare's serverless stack. That means there are no traditional "servers" we manage — Cloudflare runs our code at hundreds of locations around the world, close to whoever is shopping. The benefits: it is fast everywhere, cheap to run when traffic is low, and scales automatically when a viral product hits ten thousand orders in an hour.

Who the customers are

Two very different groups use Flomerce, and the codebase is organised around that split.

- **Store owners (also called "merchants" or "tenants").** A jewellery designer in Karwar, a clothing brand in Bengaluru, a beauty maker in Mumbai. They sign up on `flomerce.com`, pick a plan, run the site-creation wizard, and then live inside the **admin panel** to add products, edit copy, watch orders, and respond to messages.
- **Shoppers (also called "customers").** Anyone who lands on a merchant's storefront to browse and buy. They never visit `flomerce.com` directly — they only see the merchant's branded subdomain (or custom domain).

There is also a **platform owner** (you, plus future internal staff) who can see across all sites via the **owner admin** views and resolve cross-tenant issues — billing disputes, abuse, infrastructure incidents.

The product vision in one paragraph

Most small Indian businesses can't afford a Shopify Plus account and don't want the DIY pain of WordPress + WooCommerce. Flomerce sits in the gap: low subscription cost, opinionated defaults that look professional out of the box, India-first integrations (Razorpay, Shiprocket, Brevo, WhatsApp), and a single admin panel that a non-technical shop owner can actually use. The technology stack is intentionally boring and serverless so the team stays small and operating costs stay low even at scale.

How to use this guide

This is both a **personal learning guide** for the platform owner and an **onboarding handbook** for new team members. It is meant to be read top-to-bottom the first time, then dipped back into when something specific comes up.

A few promises:

- Every code snippet you see is from a real file in the codebase, with the file path shown above the snippet so you can open it and read more.
- Every diagram is hand-drawn for this guide; nothing is hidden behind links or external dashboards.
- We assume you know roughly what HTML and JavaScript are. If a concept gets fuzzy, Section 4 has plain-English refreshers and Section 9 has a 90-day study plan.

Reading paths

Different roles need different things. Pick whichever start point matches you and follow the order below — you can always read the rest later.

If you are...	Start with
The platform owner (technical refresher)	Sections 1 → 2 → 5 → 6 → 8 → 10
A new developer	Sections 2 → 4 → 5 → 6 → 7 → 9
A new support / customer-success teammate	Sections 1 → 3 → 7 → 8 → 10
A new content / SEO teammate	Sections 1 → 3 → 5 (frontend folders only) → 8 → 10
A non-technical operations teammate	Sections 1 → 3 → 7 → 10

If you finish this guide and still have a question, that is normal. The next stop is `replit.md` in the root of the repository — it is the long-form architecture log that the team updates as decisions are made.

2. The big picture

Multi-tenant SaaS, in plain English

A "tenant" is just a customer of our platform — in our case, a store owner. "Multi-tenant" means **all of those store owners share the same running software and the same infrastructure**, but their data is kept strictly separate.

Compare two ways of running this:

- **Single-tenant** (the old way): every customer gets their own server, their own database, their own deploy. Expensive, slow to update, hard to operate.
- **Multi-tenant** (Flomerce): one Cloudflare Worker, one (small) set of databases, serving every store. Each shopper request carries enough information (the subdomain) for our code to figure out which store they want and pull only that store's data.

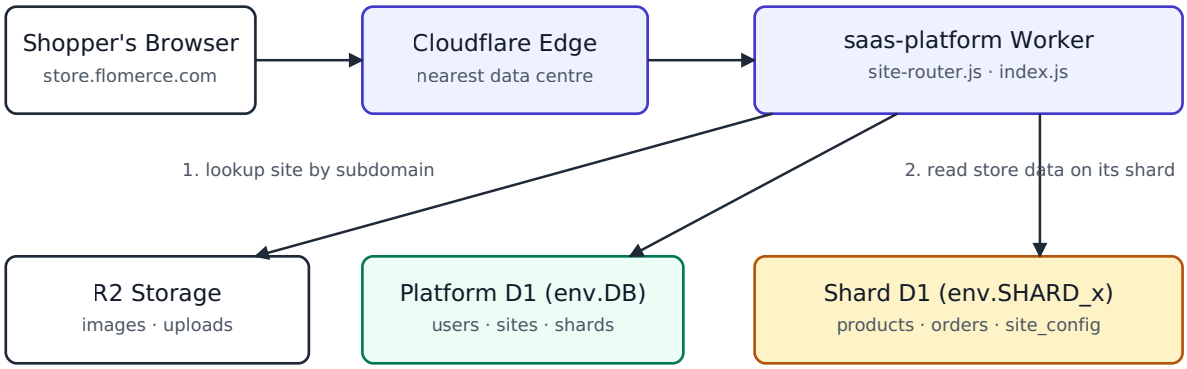
The trick to making this safe is **always tagging data with `site_id`** and never forgetting that filter. Every query against shop data looks like:

```
SELECT * FROM products WHERE site_id = ? AND is_active = 1;
```

If you ever forget the `WHERE site_id = ?`, one merchant could see another merchant's products. So every backend file in the codebase that touches store data is paranoid about including that filter — and the database utilities are designed to make it hard to skip.

How a shopper request travels

When a shopper types `acme.flomerce.com` into their browser:



Each shopper request follows this same path.

Step by step:

1. **Browser → Cloudflare edge.** Cloudflare is the doorman. The browser's request hits whichever Cloudflare data centre is closest to the shopper, anywhere in the world, in tens of milliseconds.
2. **Edge → our Worker.** Cloudflare matches the hostname to our Worker (`saas-platform`) and runs `backend/workers/index.js`. That file decides whether the request is an API call (`/api/...`), a static asset, or a storefront page.
3. **Site lookup.** For a storefront page, `site-router.js` reads the subdomain, looks it up in the **platform database** (`env.DB`) to find the matching `sites` row, and figures out which **shard database** holds that store's products.
4. **Shard read.** Products, categories, orders, and storefront config are pulled from the **shard database** (e.g. `env.SHARD_DB1`), always filtered by `site_id`.
5. **HTML response.** The Worker assembles the storefront HTML (with proper SEO meta tags injected per page) and returns it. Images come from R2 storage on demand.

The whole round-trip usually takes well under a second, and almost everything above is automatic.

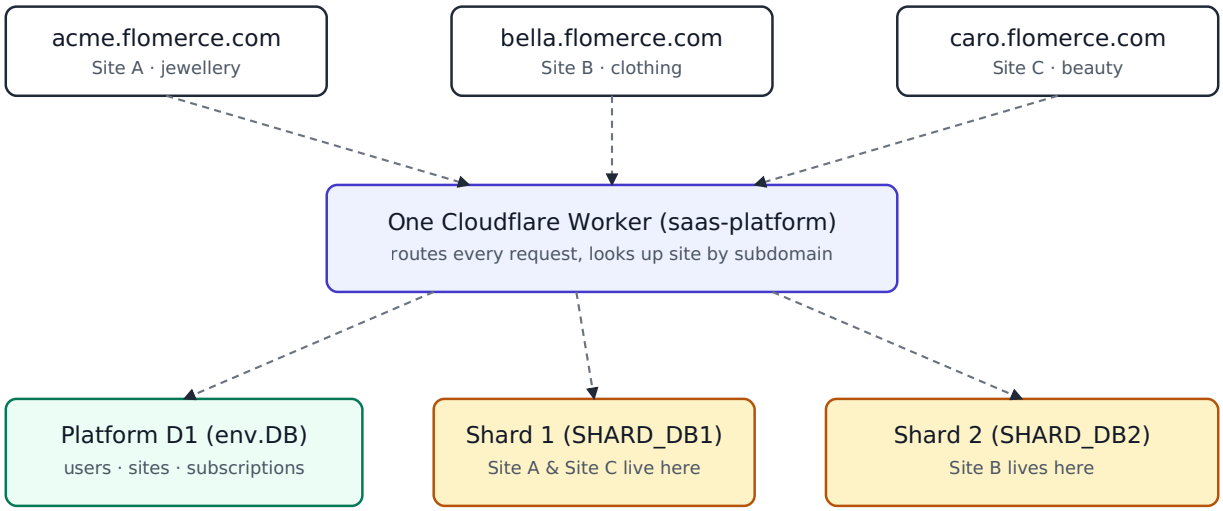
Why we split data between a "platform DB" and "shard DBs"

Cloudflare D1 (our database) has a per-database size limit. If we put every store into one giant database, we would hit the limit quickly. So we keep:

- **One platform database** (`env.DB`) for things that are global: user accounts, the `sites` directory, subscription/billing rows, the `shards` registry, and cross-site analytics roll-ups.

- **Several shard databases** (`env.SHARD_DB1` , `env.SHARD_DB2` , …), each holding the per-store tables (products, orders, site_config, …) for a slice of stores.

When a new store signs up, the platform picks an active shard with room and records that choice in `sites.shard_id` . From then on, looking up a store's data is a two-hop process: ask the platform DB which shard the store lives on, then ask that shard for the actual data.



All three stores share one Worker. Their data is split between the platform DB and one of several shards.

Same one Worker, three storefronts, two shards. Adding store #4 is mostly free: the Worker doesn't change, we just write one more row into `sites` pointing at whichever shard has space.

3. Vocabulary

A small glossary so the rest of the guide makes sense. Definitions are short on purpose. Where useful, we include a real Flomerce example.

3.1 Business / product terms

Platform. The whole Flomerce product — the landing page, the dashboard, the admin panel, the storefronts, all of it. When someone says "the platform is down," they mean the entire system, not one merchant's site.

Site (a.k.a. tenant, store, shop). One merchant's online store. Has a unique subdomain (`acme.flomerce.com`), an entry in the `sites` database table, and its own slice of data on a shard. **In code we almost always say "site"** — "tenant" only appears in architecture conversations.

Storefront. The shopper-facing part of a site. The product pages, the cart, the checkout — what a customer sees.

Admin panel. The part of a site that only the merchant (or their staff) can see. Add products, watch orders, edit homepage copy. Lives at `acme.flomerce.com/admin`.

Owner. Two meanings — context matters. Inside the codebase "owner" most often means the **store owner / merchant** (the human who created the site; authenticates against `users` table; logs into the dashboard at `flomerce.com`). Less commonly it means **the platform owner** (us — the human who runs Flomerce itself). When the distinction matters, this guide says "store owner" or "platform owner" explicitly.

Customer (a.k.a. shopper). A person buying something from a merchant. Lives in the per-store `site_customers` table on a shard, NOT in the platform `users` table.

Staff. A non-owner human the merchant has invited to help run their store. Has limited permissions inside the admin panel (e.g. can edit products but not billing).

Subscription plan. Trial → Starter → Growth → Pro → Enterprise. Each plan unlocks more features (push notifications, advanced SEO, more staff seats, more storage). Enforced both server-side (`backend/utils/usage-tracker.js`) and visually in the admin panel.

Shard. A database that holds the per-store tables for many stores. We have multiple shards so no single database gets too big. The `shards` table in the platform DB lists them all.

Custom domain. If a merchant owns `acme.com`, they can point it at their Flomerce site instead of using the default `acme.flomerce.com`. We verify ownership and route traffic through Cloudflare.

3.2 Technical terms

Frontend. Code that runs in the shopper's or merchant's browser. Built with React. Lives under `frontend/src/`.

Backend. Code that runs on Cloudflare's servers (well — workers). Receives HTTP requests, talks to the database, returns JSON or HTML. Lives under `backend/`.

API. A set of URLs that return data instead of HTML pages. Ours all start with `/api/` (e.g. `POST /api/auth/login`). The frontend calls them with `fetch`; the backend handlers live in `backend/workers/`.

Database. Where we store data on disk. We use **Cloudflare D1**, which is SQLite under the hood — a small, file-based SQL database. Tables, columns, rows.

Migration. A SQL file that changes the database structure (creates a table, adds a column). Files in `backend/migrations/` are numbered (`0001_…`, `0002_…`) so we always run them in order. New databases get every migration in sequence; existing ones skip the ones already applied.

Cloudflare Worker. A small piece of JavaScript that Cloudflare runs at the edge whenever a request comes in. Like an API server, but stateless and global. Cold-starts in milliseconds, scales to anything.

JWT (JSON Web Token). A signed piece of text the backend hands out at login. The browser stores it (in `localStorage` and a cookie) and sends it back with every later request, so the backend can recognise who is calling. We sign ours with HS256 and the `JWT_SECRET` environment variable.

Cookie. A small string the browser remembers per-site and sends with every request to that site. We use one named `auth_token` for login, set with the `HttpOnly` flag so JavaScript can't read it (prevents some attacks).

Route. A URL pattern that maps to a piece of code. On the backend, `/api/auth/login` routes to `handleLogin()`. On the frontend, React Router maps `/dashboard` to `<DashboardPage />`.

Component. A reusable piece of frontend UI. In React, a component is a JavaScript function that returns the HTML it should render. `<LoginPage />`, `<ProductCard />`, `<Footer />` are all components.

Context (React). A way for many components to share the same data without passing it manually through every level. We use contexts for the logged-in user (`AuthContext`), the cart (`CartContext`), the active store config (`SiteContext`), and a few others.

Build. The step that takes our React source files (`.jsx`) and produces the final small JavaScript bundles the browser actually downloads. We run `npm run build` inside `frontend/src/platform` and `frontend/src/storefront`.

Hash (password). A one-way scramble of the password. We never store the raw password — we store its hash and compare hashes at login. We use PBKDF2 with 100,000 rounds and a per-user salt (`backend/utils/auth.js`).

Environment variable / secret. Configuration we don't want in the code: API keys, the JWT secret, the database connection. Cloudflare stores them encrypted and exposes them as `env.SOMETHING` inside our Worker.

R2. Cloudflare's object storage (think "Dropbox for our app"). We use it for uploaded images. Images are referenced by a key like `sites/<siteId>/products/<filename>` .

4. Core concepts explained

If you have only dabbled in code before, this section gives you just enough mental model to follow the rest of the guide. None of it is Flomerce-specific — these are foundations. Section 9 has links to deeper external resources.

4.1 HTML, CSS, and JavaScript

Every web page is built from three layers:

- **HTML** is the structure: headings, paragraphs, lists, forms, buttons. It says what is on the page.
- **CSS** is the appearance: colours, fonts, spacing, layout. It says how it looks.
- **JavaScript** is the behaviour: clicks, fetches, animations, validation. It says what happens when the user does something.

Open any web page, hit `F12`, and you can see all three layers in the browser's developer tools. We will use that a lot in Section 7.

A trivial example:

```
<button id="say-hi" style="background:#111; color:#fff;">Click me</button>
<script>
  document.getElementById('say-hi').addEventListener('click', () => {
    alert('Hi from JavaScript');
  });
</script>
```

The button is HTML, its colours are CSS, and the alert is JavaScript.

4.2 What React is, and what a component looks like

Writing every page by hand-stitching HTML and JavaScript gets unwieldy fast. **React** is a library that lets us describe UI as small, reusable functions. Each function returns what looks like HTML (technically "JSX") and React handles turning that into the real page in the browser.

Here is a tiny component, similar to what you will see in `frontend/src/...`:

```
import { useState } from 'react';

function CounterButton() {
  const [count, setCount] = useState(0);
  return (
    <button onClick={() => setCount(count + 1)}>
      Clicked {count} times
    </button>
  );
}
```

A few ideas to anchor:

- `useState` is a **React hook** that gives the component a memory cell. When you call `setCount`, React re-renders the component with the new value.
- The return value (`<button>...</button>`) is JSX. It compiles to plain JavaScript at build time.
- The component can be reused: `<CounterButton />` in any page and you have a fresh counter.

Real components in our codebase look like `LoginPage.jsx`, `ProductCard.jsx`, `AdminSidebar.jsx`. They get bigger but the shape is the same.

4.3 What an API call is

When a frontend needs data from the backend (or wants to save data), it makes an **HTTP request** to a URL. The browser's built-in `fetch` function does this. Roughly:

```
const res = await fetch('https://flomerce.com/api/auth/login', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ email, password }),
});
const data = await res.json();
```

Every API call has:

- a **method** (`GET` reads, `POST` creates, `PUT` updates, `DELETE` removes),
- a **URL** (which endpoint),
- optional **headers** (auth token, content type),
- an optional **body** (the JSON payload), and
- a **status code** in the response (`200` ok, `401` not logged in, `500` server crashed). You will see these constantly in the Network tab in Section 7.

4.4 What a database is

A database is a structured place to store data on disk so it survives restarts. We use **SQLite** (via Cloudflare D1). Data lives in **tables**; each table has **columns** (like fields on a form) and **rows** (one per record).

Two examples from our schema:

```
-- backend/migrations/0001_initial_schema.sql
CREATE TABLE users (
  id TEXT PRIMARY KEY,
  email TEXT UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  name TEXT NOT NULL,
  email_verified INTEGER DEFAULT 0,
  created_at TEXT DEFAULT (datetime('now'))
);
```

You read and write rows with **SQL**:

```
SELECT id, email FROM users WHERE email = 'alice@example.com';
INSERT INTO users (id, email, name, password_hash) VALUES (?, ?, ?, ?);
UPDATE users SET name = ? WHERE id = ?;
DELETE FROM sessions WHERE user_id = ?;
```

The `?` placeholders are filled in safely by our code (`stmt.bind(value)`), which prevents a class of bug called SQL injection.

Why we have two kinds of databases. As covered in Section 2: the **platform DB** holds global tables (users, sites, subscriptions); each **shard DB** holds per-store tables (products, orders, site_config, ...). The split is purely about keeping each individual D1 file small enough.

4.5 What "Cloudflare Workers" actually means

Forget traditional servers. A **Cloudflare Worker** is a small bundle of JavaScript that Cloudflare runs **on demand** when a request comes in, in hundreds of locations worldwide. It boots in single-digit milliseconds, runs your handler, and exits.

Implications you will feel daily:

- **No long-running processes.** No background daemons (we use cron triggers instead). No in-memory caches that survive between requests reliably.
- **Cold-start friendly.** Even if no one has hit the worker for hours, the next request is fast.

- **Bindings.** Instead of opening a database connection, our code accesses `env.DB`, `env.SHARD_DB1`, `env.STORAGE`, etc. These are pre-wired by Cloudflare based on `wrangler.toml`.
- **One worker for everything.** Our entire backend is the single `saas-platform` Worker. The router in `backend/workers/index.js` decides which handler to invoke for each path.

If this sounds limiting, it is — and that is the point. The constraints force a clean, small architecture that is hard to break and easy to scale.

5. Guided tour of the codebase

Open the project in any editor. The folders worth knowing about are listed below, in roughly the order you will visit them. For each folder we describe **what lives there**, **when you would open it**, and **one example file to peek at first** so it doesn't feel like a wall of unknown filenames.

5.1 `backend/workers/`

What lives there. Every backend HTTP handler. The single Worker entry point is `backend/workers/index.js`; it routes API paths to handlers that live in two subfolders:

- `backend/workers/platform/` — handlers for things scoped to the platform itself: authentication (`auth-worker.js`), site management (`sites-worker.js`), platform-wide payments and subscriptions (`payments-worker.js`), the platform admin (`admin-worker.js`), email delivery (`email-worker.js`), translations (`i18n-worker.js`), and billing (`billing-worker.js`).
- `backend/workers/storefront/` — handlers for per-store features that shoppers and admins use: products, orders, cart, wishlist, customer auth, reviews, blog, file uploads, analytics, notifications, shipping, the per-site admin panel, etc. The names match: `products-worker.js`, `orders-worker.js`, and so on.
- `backend/workers/seo/` — server-side SEO injection: meta tags, sitemap generation, robots.txt, and structured data (Product / Article schema).
- `backend/workers/site-router.js` — the routing brain that reads the request's hostname/subdomain, finds the matching `sites` row, and renders the storefront HTML (with SEO injected). All non-`/api/` requests go through here.

When you would open it. Whenever a `/api/...` URL behaves wrong, or whenever you want to know how a specific storefront URL gets served.

Peek at first. `backend/workers/index.js` — the top of that file is a giant `switch (resource)` block that maps every URL prefix to its handler. It's the single best mental map of the backend.

5.2 `backend/utils/`

What lives there. Helpers shared by many handlers. The important ones:

- `auth.js` — password hashing (PBKDF2), JWT sign/verify, the `validateAuth` function that every protected handler calls.

- `site-db.js` — `resolveSiteDB(env, site)` and friends. **This is how the backend gets a handle on the right shard database for a given site.** Always use it; never hard-code a shard binding.
- `db-init.js` — `ensureTablesExist(env)`. Idempotent table/column creation that runs at the top of every Worker invocation. Belt-and-suspenders against shards that pre-date a migration.
- `crypto.js` — AES-GCM encrypt/decrypt for secrets we store in the database (e.g. each merchant's Microsoft Translator key).
- `email.js` — sends transactional emails via Brevo (formerly Sendinblue): verification links, order confirmations, password resets.
- `helpers.js` — `jsonResponse`, `errorResponse`, `corsHeaders`, `generateId`, `validateEmail`, `sanitizeInput`. Tiny pure functions.
- `usage-tracker.js` — counts D1 / R2 bytes per site, enforces plan limits, and exposes the `checkFeatureAccess` and `checkCountLimit` helpers used to gate paid features.
- `cache.js` and `cdn-cache.js` — Workers Cache API helpers and Cloudflare CDN edge purge.
- `shiprocket.js`, `web-push.js`, `whatsapp.js` — wrappers around external services. Self-contained on purpose so swapping providers is local.

When you would open it. When you need a primitive (hash a password, send an email, find the right shard) or want to understand a specific cross-cutting concern.

Peek at first. `backend/utils/auth.js` — short, well-commented, and you'll need to understand it for Section 6.

5.3 `backend/migrations/`

What lives there. Numbered `.sql` files describing every change to the database schema since day one (`0001_initial_schema.sql` through `0022_customer_language_prefs.sql` and beyond). Each one is meant to be run exactly once, in order. The platform-DB tables also have idempotent `CREATE TABLE IF NOT EXISTS` mirrors inside `db-init.js` so a fresh deploy self-heals even if someone forgot to apply a migration.

When you would open it. Whenever you want to know "what columns does this table actually have?" without going to the live database. Or when you are adding a new column.

Peek at first. `0001_initial_schema.sql` (the foundation) and `0003_auth_tables.sql` (sessions and password resets — what we'll use in Section 6).

5.4 `frontend/src/platform/`

What lives there. The React app behind `flomerce.com` itself: the marketing landing page, signup / login / verify-email / reset-password pages, the dashboard where store owners pick a plan and create / manage their sites, the "owner admin" view (cross-tenant tools for the platform owner — us), and the legal pages (Terms, Privacy, Refund, Shipping). Built with Vite. Output goes to `frontend/` at the project root.

Inside `frontend/src/platform/src/` you'll see the standard React layout:

- `pages/` — top-level route components (`LoginPage.jsx` , `DashboardPage.jsx` , `OwnerAdminPage.jsx` , ...).
- `components/` — reusable bits (`Navbar.jsx` , `LandingPricing.jsx` , `SiteCreationWizard.jsx` , ...).
- `services/` — API wrappers (`api.js` , `authService.js` , `siteService.js` , `paymentService.js`).
- `context/` — `AuthContext.jsx` (the logged-in user across the whole platform app).

When you would open it. Whenever something on `flomerce.com` itself looks wrong — login, signup, dashboard, plan upgrade flow, the site-creation wizard.

Peek at first. `pages/LoginPage.jsx` — small, complete, and we walk through it line-by-line in Section 6.

5.5 `frontend/src/storefront/`

What lives there. The React app that runs on every merchant's subdomain. Same `pages/components/services/context` layout as the platform app, plus an `admin/` subfolder for the per-store admin panel pages.

Highlights:

- `pages/HomePage.jsx` , `ProductDetailPage.jsx` , `CategoryPage.jsx` , `CartPage.jsx` , `CheckoutPage.jsx` — the shopper-facing pages.
- `pages/AdminPanel.jsx` — wraps every per-store admin route (`/admin/*`) including login, dashboard, products, orders, settings, and the visual customizer.
- `components/templates/modern/` — Modern-theme variants of shared components (Navbar, Footer, ProductCard, Hero, etc.). The Classic theme uses the default components in `components/` . The Theme switch is decided by `ThemeContext` reading `site_config.settings.theme` .
- `context/SiteContext.jsx` — fetches `/api/site` once, holds the active store's config (logo, colours, plan, settings) for every component to consume via `useSite()` .

When you would open it. Whenever something looks wrong on a merchant's storefront, or when you are tweaking how the admin panel behaves.

Peek at first. `pages/HomePage.jsx` — gives you a feel for how a storefront is composed from sections (`<Hero />` , `<FeaturedCategories />` , `<TrendingNow />` , ...).

5.6 `frontend/src/shared/`

What lives there. Code reused by both the platform app and the storefront app — lives in its own folder so neither side accidentally couples to the other's internals.

- `shared/ui/` — generic dialog primitives: `Modal` , `AlertModal` , `ConfirmDialog` , `Toast` , plus `styles.css` .
- `shared/i18n/` — `LanguageSwitcher.jsx` , the `i18next` init code (`init.js`), and the source-of-truth English JSON catalogs under `locales/en/` . Only the five System-A landing surfaces actually use these (see `replit.md` → Translation Architecture).

When you would open it. Adding a new modal/toast for use across both apps, or adding/editing an English string used on the landing page or shopper language switcher.

Peek at first. `shared/ui/Modal.jsx` — small, demonstrates how we keep common UI bits framework-light.

6. Trace your first feature: User Login

This is the headline section of the guide. We will follow exactly what happens when a store owner logs in at `flomerce.com/login` — every file involved, every function call, every database touch — and end with a sequence diagram you can keep in your head.

The feature we are tracing is **platform user login** (the store owner). Shopper login on a storefront uses a different code path (`customer-auth-worker.js`) that mirrors this one closely; once you understand this walk-through, that one will read identically.

6.1 What the user sees

Two fields and a button. The form lives in `frontend/src/platform/src/pages/LoginPage.jsx`:

```
// frontend/src/platform/src/pages/LoginPage.jsx (trimmed)
const { isAuthenticated, login, loading } = useAuth();
const navigate = useNavigate();

async function handleSubmit(e) {
  e.preventDefault();
  setError('');
  setSubmitting(true);
  try {
    const result = await loginService(email, password);
    if (result.token) {
      login(result.token, result.user);
      navigate('/dashboard');
    } else if (result.error?.includes('verify')) {
      setError("Please verify your email before logging in.");
    } else {
      throw new Error(result.error || "Login failed");
    }
  } catch (err) {
    setError(err.message || "Login failed");
  } finally {
    setSubmitting(false);
  }
}
```

What this is doing in plain English:

1. Pull the `email` and `password` out of the form's state.
2. Call `loginService(...)` (which lives in `services/authService.js`) — that's the function that actually makes the HTTP call.
3. If the backend returned a `token`, store it via `login(token, user)` (more on that in 6.4) and navigate to `/dashboard`.
4. If anything went wrong, show a red error message and stop.

The component does **not** know about `fetch`, JWTs, cookies, or the database. That separation — UI components don't talk HTTP directly — is consistent across the codebase.

6.2 The frontend service layer

`authService.js` is a thin wrapper. Its job is to standardise URLs and method/body shape for every auth-related endpoint. Open it and you'll see almost identical 3-line functions for `login`, `signup`, `logout`, `verifyEmail`, `requestPasswordReset`, etc.

```
// frontend/src/platform/src/services/authService.js
import { apiRequest } from './api.js';

export async function login(email, password) {
  return apiRequest('/api/auth/login', {
    method: 'POST',
    body: JSON.stringify({ email, password }),
  });
}
```

`apiRequest` (in `services/api.js`) is the single place that knows about our auth header convention, our error class, and the `/api` base URL:

```
// frontend/src/platform/src/services/api.js (trimmed)
export async function apiRequest(endpoint, options = {}) {
  const url = `${API_BASE_URL}${endpoint}`;
  const token = getToken();          // localStorage.getItem('auth_token')

  const headers = {
    'Content-Type': 'application/json',
    ...options.headers,
  };
  if (token) headers['Authorization'] = `Bearer ${token}`;
  headers['X-Session-ID'] = getSessionId();

  const response = await fetch(url, { ...options, headers, mode: 'cors',
                                      credentials: 'omit' });

  const data = await response.json();
  if (!response.ok) {
    const errorMsg = data.message || data.error || 'Request failed';
    throw new APIError(errorMsg, response.status, data.code);
  }
  return data;
}
```

A few details worth noting:

- We send the token in an **`Authorization: Bearer ...`** header. We do **not** use the cookie for our own frontend (that's why `credentials: 'omit'`). The cookie is set anyway for older clients and will become useful if we ever add server-side rendering.

- Errors are thrown as a custom `APIError` class so callers can `try/catch` and branch on `err.code` ("`USE_GOOGLE_LOGIN`", "`EMAIL_NOT_VERIFIED`", etc.).

6.3 The backend handler

The request lands at the Worker's `fetch` entrypoint (`backend/workers/index.js`), which routes any path under `/api/` to `handleAPI`, which routes `auth` to `handleAuth` in `backend/workers/platform/auth-worker.js`:

```
// backend/workers/platform/auth-worker.js (trimmed)
async function handleLogin(request, env) {
  await ensureAuthTables(env);
  const { email, password } = await request.json();

  const user = await env.DB.prepare(
    'SELECT id, email, password_hash, name, email_verified FROM users WHERE email = ?'
  ).bind(email.toLowerCase()).first();

  if (!user) return errorResponse('Invalid email or password', 401,
'INVALID_CREDENTIALS');
  if (!user.password_hash)
    return errorResponse('This account uses Google sign-in.', 401, 'USE_GOOGLE_LOGIN');

  const isValid = await verifyPassword(password, user.password_hash);
  if (!isValid)         return errorResponse('Invalid email or password', 401,
'INVALID_CREDENTIALS');
  if (!user.email_verified) return errorResponse('Please verify your email', 401,
'EMAIL_NOT_VERIFIED');

  const token = await generateJWT({ userId: user.id, email: user.email },
                                  env.JWT_SECRET || 'your-secret-key');

  // single-session: drop any older session rows for this user
  await env.DB.prepare('DELETE FROM sessions WHERE user_id = ?').bind(user.id).run();

  const sessionId = generateId();
  await env.DB.prepare(
    `INSERT INTO sessions (id, user_id, token, expires_at) VALUES (?, ?, ?, ?)`
  ).bind(sessionId, user.id, token, getExpiryDate(24 * 7)).run();

  const response = successResponse({
    user: { id: user.id, email: user.email, name: user.name,
            emailVerified: !!user.email_verified },
    token,
  }, 'Login successful');

  response.headers.set('Set-Cookie',
    `auth_token=${token}; Path=/; HttpOnly; SameSite=Strict; Max-Age=${60 * 60 * 24 *
7}`);
  return response;
}
```

Walking through it:

1. **Ensure auth tables exist.** `ensureAuthTables` is paranoid — `CREATE TABLE IF NOT EXISTS sessions / email_verifications / password_resets`. Cheap, idempotent, and saves us from "the migrations were never run" incidents.
2. **Look up the user by email.** Lower-casing the email keeps the table case-insensitive without needing a SQL collation. If no row, we deliberately return the same error as a wrong password (`INVALID_CREDENTIALS`) so attackers can't use the error message to enumerate accounts.
3. **Reject Google-only accounts.** If the user signed up with Google originally they have no `password_hash`. We reply with a specific code so the frontend can suggest "log in with Google".
4. **Verify the password.** `verifyPassword` is in `backend/utils/auth.js` — PBKDF2-SHA256 with 100k rounds and the per-user salt that's stored alongside the hash. If the recomputed hash matches what's in the DB, the password is right.
5. **Block unverified accounts.** Email verification is mandatory. The front-end recognises `EMAIL_NOT_VERIFIED` and shows a friendly resend prompt.
6. **Mint a JWT.** `generateJWT` (also in `auth.js`) HMAC-signs a tiny payload (`{ userId, email, iat, exp }`) with `env.JWT_SECRET`. The token is just a string — three base64 chunks separated by dots.
7. **Single-session policy.** We delete any prior session rows for this user. Logging in on a new device kicks out the old device. (You may want to make this multi-session in the future — that's a one-line change.)
8. **Record the session.** A new row in `sessions` holds the JWT and an expiry one week out. This row is what gets cleared when the user (or password reset) logs them out.
9. **Set the cookie.** `auth_token=...; HttpOnly; SameSite=Strict; Max-Age=604800`. `HttpOnly` means JavaScript on the page can't read it — that's a defence against XSS attacks.
10. **Return the token + user in the JSON body.** The frontend reads the body and stores the token in `localStorage` for future `Authorization` headers.

6.4 What the frontend does after a successful login

Back in `LoginPage.jsx`, `login(token, user)` calls into `AuthContext.jsx`:

```
// frontend/src/platform/src/context/AuthContext.jsx (trimmed)
const saveToken = useCallback((newToken) => {
  if (newToken) localStorage.setItem('auth_token', newToken);
  else          localStorage.removeItem('auth_token');
  setToken(newToken);
}, []);

const login = useCallback((tokenValue, userData) => {
  saveToken(tokenValue);
  setUser(userData);
}, [saveToken]);
```

Two things just happened:

- The JWT is in `localStorage` under `auth_token`. Every future `apiRequest` reads it via `getToken()` and adds the `Authorization: Bearer ...` header automatically.
- The React `user` state is set, so `isAuthenticated` becomes `true`. Any page with `useEffect(() => { if (isAuthenticated) navigate('/dashboard') }, ...)` immediately bounces. The component itself also calls `navigate('/dashboard')` to be explicit.

6.5 How later requests prove the user is logged in

On every reload, `AuthProvider` runs:

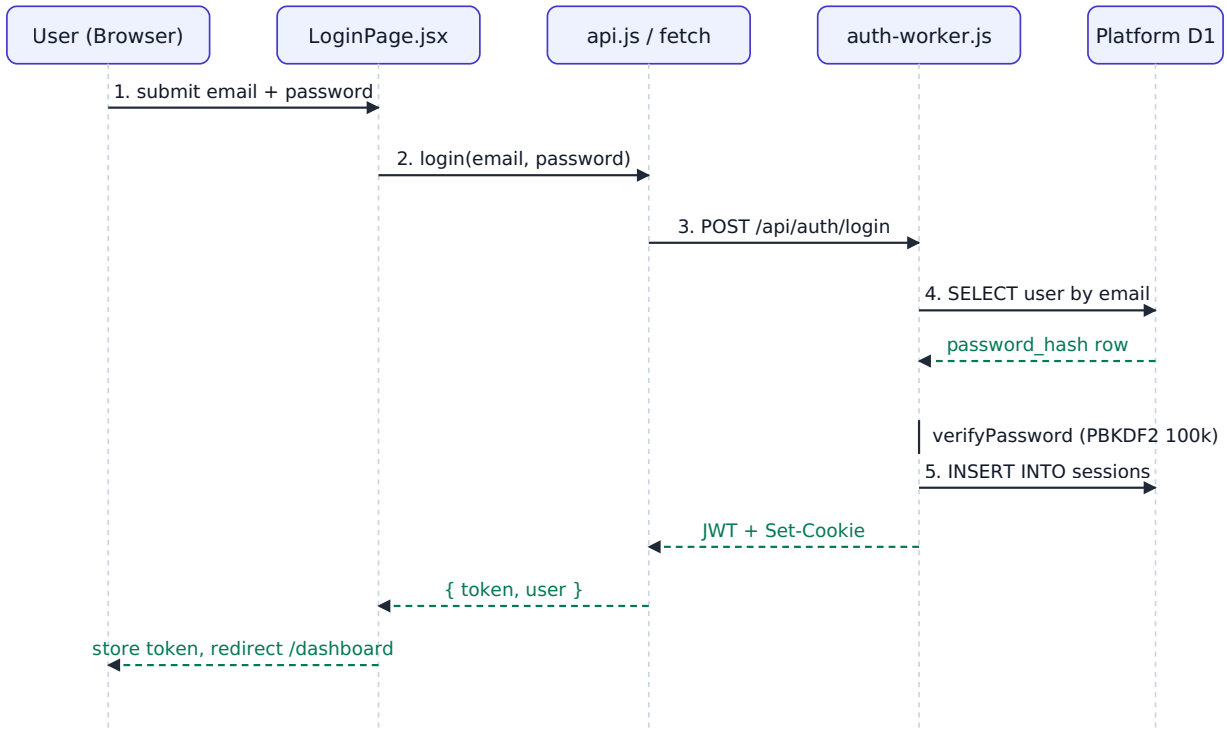
```
useEffect(() => {
  async function loadUser() {
    if (!token) { setLoading(false); return; }
    try {
      const data = await getProfile();         // GET /api/auth/me
      setUser(data.user || data);
    } catch (e) {
      saveToken(null); setUser(null);          // expired/invalid → log out
    } finally { setLoading(false); }
  }
  loadUser();
}, [token, saveToken]);
```

`/api/auth/me` reaches `handleGetCurrentUser` in `auth-worker.js`, which calls `validateAuth(request, env)` from `backend/utils/auth.js`. That function:

1. Reads the `Authorization: Bearer ...` header (or falls back to the `auth_token` cookie).
2. Calls `verifyJWT` to check the HMAC signature with `env.JWT_SECRET` and confirm the `exp` (expiry) claim is in the future.
3. Re-loads the `users` row to return up-to-date `name` / `email_verified`.

If any step fails, `validateAuth` returns `null` and the handler responds 401. The frontend treats 401 as "log this user out", clears `localStorage`, and sends them back to the login page.

6.6 The whole login, in one diagram



Every subsequent request the user makes carries the JWT, so the server doesn't need to re-check the password — just verify the signature on the token. That's the entire point of using JWTs.

6.7 Two related flows in one paragraph each

Forgot password. The "Forgot password?" link opens a modal that calls `requestPasswordReset(email)` → `POST /api/auth/request-reset` → `handleRequestReset` in `auth-worker.js`. That handler creates a one-hour token in `password_resets`, sends an email via `email-worker.js` → Brevo, and returns success even if the email doesn't exist (so attackers can't enumerate accounts). Clicking the email link lands on `ResetPasswordPage.jsx` which posts to `/api/auth/reset-password`.

Sign up with Google. The "Sign in with Google" button is rendered by Google's GIS script. After the user picks an account, Google calls our `handleGoogleResponse` callback with a credential JWT. We post it to `/api/auth/google`, which validates the token against `oauth2.googleapis.com/tokeninfo`, checks the audience matches our `GOOGLE_CLIENT_ID`, then either creates a passwordless `users` row (with `email_verified = 1` since Google has already verified) or logs an existing user in. From there, the rest of the flow is identical to password login.

7. The debugging playbook

When something looks broken, the fastest path to a fix is **(a) seeing the error**, **(b) telling someone clearly what you saw**, and **(c) knowing roughly where to look in the code**. This section gives you all three.

7.1 How to read an error message

A typical browser error looks like:

```
TypeError: Cannot read properties of undefined (reading 'name')
    at ProfilePage (ProfilePage.jsx:42:18)
    at renderWithHooks (react-dom.development.js:14803)
    ...
```

The bits that matter:

- **The first line** describes what went wrong. "`Cannot read properties of undefined`" almost always means we tried to access `something.name` but `something` was `undefined`. Maybe the data hadn't loaded yet.
- **The first line that points to OUR code** (here, `ProfilePage.jsx:42`) tells you where to look. Skip everything below that — those are React's internals.
- **Click the file:line link** in the browser. DevTools jumps straight to the source.

For backend errors, the message is in the JSON response body or the Cloudflare Worker logs. We always include a short `error` string and a stable `code` (e.g. `INVALID_CREDENTIALS`, `EMAIL_SEND_FAILED`).

7.2 The browser DevTools, in 90 seconds

Press **F12** (or `Cmd+Option+I` on a Mac). You get a panel with several tabs. The two you'll live in are:

Console. A live log. Every `console.log(...)` call from the frontend, every unhandled error, every warning. Red entries are errors; yellow entries are warnings. Click the file:line link to jump to source.

```
+-----------------------------------------------------------+
| Console                                        [Filter: ] |
|-----------------------------------------------------------|
| [ERROR] Failed to load resource: 401                      |
|         flomerce.com/api/auth/me                          |
| [LOG]   user logged in: alice@example.com                 |
+-----------------------------------------------------------+
```

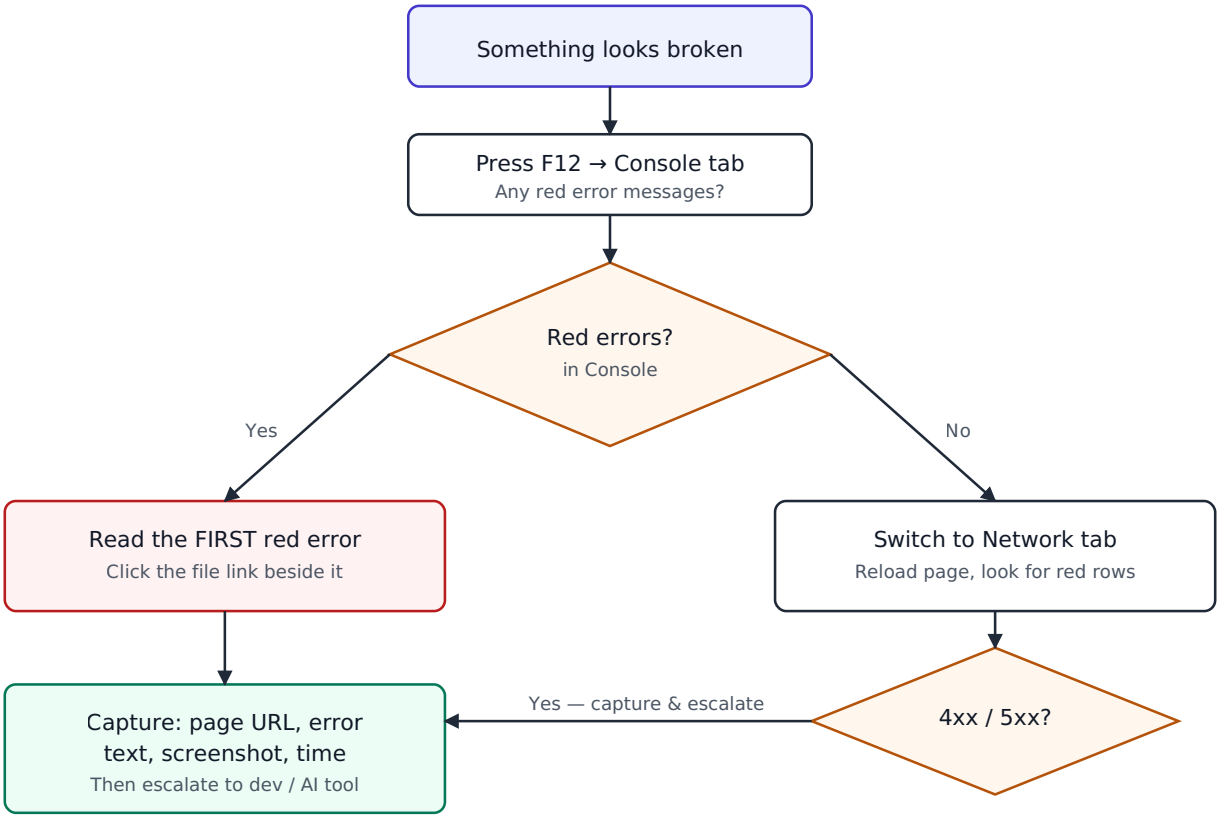
If you see a **red** entry, that is almost always your starting point.

Network. Every HTTP request the page made. Each row shows the URL, the status (`200` , `401` , `500`), the size, and how long it took. Click any row to see request headers, request body, response headers, response body.

```
+-----------------------------------------------------------------+
| Network                        [XHR | JS | CSS | Img | All]    |
|-----------------------------------------------------------------|
| Name                Status  Type        Size   Time            |
| /api/auth/login     200     fetch       412B   124ms           |
| /api/auth/me        401     fetch        86B    98ms <-- bad   |
| logo.png            200     img         12 KB   18ms           |
+-----------------------------------------------------------------+
```

If the API call is red (`4xx` or `5xx`), open it and check the **Response** tab — the backend almost always returned a useful JSON `error` you can paste into a support ticket.

7.3 "Something broke — what do I check first?"



In words:

1. Open the page where it broke.
2. Open DevTools (`F12`) → Console.
3. **Red errors?** Read the first one. Click its file link. Even if you can't fix the code, copy the error text into your bug report.
4. **No console errors?** Switch to Network. Reload. Look for red rows. Open the first one. Note its URL, status code, and Response body.
5. **Still nothing?** It's probably backend logs. Note the time and the page URL and escalate.

7.4 How to describe a bug clearly

A good bug report has five things. AI tools and developers can usually fix it with these:

- **What you did.** "I clicked Save on the product Add page."
- **What you expected.** "The product should appear in the list."
- **What actually happened.** "I got a red banner that said 'Network error'."
- **Where.** Page URL + the merchant subdomain (or `flomerce.com`).

- **Evidence.** A screenshot of the page, the Console error text, and (ideally) the failing Network request's URL + status + Response body.

Bonus: **the timestamp.** Cloudflare logs are time-bounded, so "around 14:32 IST today" makes log digging much faster.

7.5 Common error types and what they usually mean

You see...	Usually means	First place to check
`Cannot read properties of undefined`	Data hadn't loaded yet, or an API returned a different shape.	The component named in the stack trace.
`401 Unauthorized` on an API call	The user's token is missing or expired.	`AuthContext` is logging the user out; ask them to log in again.
`403 Forbidden` on an API call	Logged in but not allowed (e.g. a feature is gated to a higher plan).	`usage-tracker.js` and `checkFeatureAccess`.
`404 Not Found` on a storefront URL	Wrong subdomain, deleted site, or unrouted path.	`site-router.js` for the lookup; the `sites` table for the row.
`500 Internal Server Error`	Backend crashed.	Cloudflare Worker logs for the timestamp + path.
`CORS error` in the console	The frontend is calling an origin the backend doesn't allow.	`corsHeaders()` in `backend/utils/helpers.js`.
White / blank page	The React bundle failed to load, or a JS error crashed render.	Console (look for the first red error) + Network (look for any 4xx on a `.js` file).
Image broken / 404	The R2 key doesn't exist or `site_media` lost its row.	`upload-worker.js` and the `site_media` / R2 prefix `sites/<id>/`.

7.6 When to escalate (and to whom)

- **Anything affecting payments** (Razorpay), **mass storefront downtime**, or **suspected data leak across tenants** → page the platform owner immediately with the timestamp and a Cloudflare log link if you have one.
- **One merchant complaining their site is broken** → confirm with the steps above, capture evidence, then escalate. If the rest of the platform is fine, it's almost always a per-site config issue (custom domain, plan expiry, shard locked for migration).

- **A whole feature looks wrong** (e.g. checkout button does nothing across all merchants) → escalate fast. We always prefer "false alarm, sorry" over "found out 6 hours later."

8. Third-party services cheat sheet

Flomerce wires together a handful of external services so we don't have to build them ourselves. For each one: **what it does**, **where it's wired up in the code**, and **where to look first if it's failing**.

Razorpay — Payments

What it does. Collects payments from shoppers (Setup B: each merchant plugs in their own Razorpay keys, money flows to their account) and from merchants themselves (Setup A: monthly subscription billing for the platform's plans, money flows to us).

Wired up at. `backend/workers/platform/payments-worker.js` (subscriptions + the per-tenant storefront webhook handler at `/api/webhooks/razorpay/:siteId`); `backend/workers/storefront/orders-worker.js` (order creation triggers the per-merchant Razorpay flow). Frontend call site is `frontend/src/storefront/src/pages/CheckoutPage.jsx`.

Failing? Check first. The Razorpay dashboard for the relevant account (merchant's or ours), then the `payment_transactions` table in the platform DB for the matching `razorpay_order_id`, then the `processed_webhooks` table to see if the webhook fired.

Brevo (formerly Sendinblue) — Transactional email

What it does. Sends every transactional email: signup verification, password reset, order confirmation to shopper, order notification to merchant, abandoned-cart reminder.

Wired up at. `backend/utils/email.js` (the HTTP wrapper around Brevo's API) and `backend/workers/platform/email-worker.js` (the route handler that templates each email type). Subject lines and bodies are translated via `backend/utils/email-i18n.js`.

Failing? Check first. The Brevo dashboard (recent activity / errors), then the Worker logs at the timestamp the email should have gone out — every email attempt logs success or the Brevo error. Make sure the Brevo API key (`BREVO_API_KEY`) is still valid.

Shiprocket — Shipping

What it does. Books shipments for orders, fetches courier rates, processes shipping webhooks (status changes propagate back to our `orders` table), and pushes notifications when an order is shipped or delivered.

Wired up at. `backend/utils/shiprocket.js` (the HTTP wrapper), `backend/workers/storefront/shipping-worker.js` (the route handler exposing shipping endpoints), and the webhook handler at `/api/webhooks/shiprocket/:siteId` (registered from `index.js`). Notification side-effects live in `backend/utils/shiprocket-notifications.js`.

Failing? Check first. The merchant's Shiprocket account is active and the API token in their site settings (`sites.shiprocket_token_encrypted` , decrypted via `backend/utils/crypto.js`) hasn't expired.

Google OAuth — Sign in with Google

What it does. Lets users (both store owners and shoppers) sign up / log in with their Google account instead of a password.

Wired up at. Frontend: the GIS script is loaded inside `LoginPage.jsx` and `SignupPage.jsx` for the platform side, and inside the storefront's `LoginPage.jsx` for shoppers. Backend: `handleGoogleLogin` in `auth-worker.js` (platform) and the equivalent in `customer-auth-worker.js` (shoppers).

Failing? Check first. The Google Cloud console for the OAuth client — make sure the `flomerce.com` and storefront domains are still in the authorised JavaScript origins. Confirm `GOOGLE_CLIENT_ID` matches on both sides.

Web Push (VAPID) — Browser push notifications

What it does. Lets a merchant send push notifications to shoppers who opted in, both manually (one-off campaign) and automatically (e.g. abandoned cart reminder).

Wired up at. `backend/utils/web-push.js` (the VAPID signing + delivery logic), `backend/workers/storefront/notifications-worker.js` (the routes), and `frontend/src/storefront/public/sw.js` (the service worker that the shopper's browser registers to receive pushes).

Failing? Check first. That `VAPID_PUBLIC_KEY` and `VAPID_PRIVATE_KEY` are set in env. That the shopper actually granted notification permission (their browser will tell them). That the push subscription row in the per-shard `push_subscriptions` table still exists.

WhatsApp Business — Order updates

What it does. Sends WhatsApp messages to shoppers for order confirmations and shipment updates, where enabled.

Wired up at. `backend/utils/whatsapp.js`. Triggered from the orders and shipping workers.

Failing? Check first. The Meta Business Suite for the merchant's WhatsApp Business account, the API token, and the approved message templates. WhatsApp templates need pre-approval — if you change the wording, expect a 24h review.

9. 90-day learning roadmap

This is a recommended 12-week study plan for a new technical hire (or a non-developer owner who wants to read code confidently). Each week is roughly 4–6 hours of study + reading the codebase. Adjust freely.

Phase 1 — Web fundamentals (weeks 1–3)

Week 1: HTML & CSS basics. Goal: read any web page's structure and styling.

- MDN's "Learn HTML" track — first 6 modules. https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Structuring_content
- MDN's "Learn CSS" track — first 4 modules. https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Styling_basics
- Codebase: open `frontend/src/storefront/src/styles/` and skim. Recognise what each rule is targeting.

Week 2: JavaScript basics. Goal: read JavaScript and follow what a function does.

- MDN "JavaScript First Steps" + "Building blocks" + "Objects". https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting
- Eloquent JavaScript chapters 1–4 (free online). https://eloquentjavascript.net/
- Codebase: read `backend/utils/helpers.js` end to end. Tiny, complete.

Week 3: How the web works. Goal: understand what `fetch`, HTTP, JSON, and status codes mean.

- MDN "HTTP overview" and "An overview of HTTP". https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview
- MDN "Using Fetch". https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
- Codebase: read `frontend/src/platform/src/services/api.js` and `authService.js`. You should be able to explain what every line does.

Phase 2 — React + the frontend (weeks 4–6)

Week 4: React basics. Goal: read a component and understand props and state.

- React's official "Learn React" tutorial (free). https://react.dev/learn
- Codebase: read `LoginPage.jsx` and `AuthContext.jsx`. Match every line to Section 6 of this guide.

Week 5: React Router & forms. Goal: understand how the URL maps to components and how form data is captured.

- React Router docs, "Tutorial". https://reactrouter.com/en/main/start/tutorial
- Codebase: open `frontend/src/platform/src/App.jsx` and find every `<Route />`. Make a list.

Week 6: Storefront tour. Goal: feel comfortable opening any storefront page and navigating to its parts.

- Codebase: trace one product page end-to-end. Start at `pages/ProductDetailPage.jsx`. Note every API call it makes; find each endpoint in `backend/workers/storefront/`.

Phase 3 — Backend & databases (weeks 7–9)

Week 7: SQL basics. Goal: read SELECT/INSERT/UPDATE statements confidently.

- "SQLBolt" interactive tutorial (free, fast). https://sqlbolt.com/
- Codebase: read `backend/migrations/0001_initial_schema.sql` and `0003_auth_tables.sql` and the `db-init.js` ensure-block.

Week 8: Cloudflare Workers + D1. Goal: understand bindings, the request model, and how D1 statements work.

- Cloudflare Workers "Get started". https://developers.cloudflare.com/workers/get-started/guide/
- Cloudflare D1 "Get started". https://developers.cloudflare.com/d1/get-started/
- Codebase: read `backend/workers/index.js` (the routing brain) and `auth-worker.js` (the handler you already understand from Section 6).

Week 9: Multi-tenant patterns. Goal: be confident about why every shard query is filtered by *`site_id`* and how the platform DB / shard split works.

- Re-read this guide's Section 2 + the `replit.md` "Database Architecture" section + `DATABASE_ARCHITECTURE.md`.
- Codebase: read `backend/utils/site-db.js` end to end. Trace one storefront route from `site-router.js` through `resolveSiteDB` to a real shard query in, say, `products-worker.js`.

Phase 4 — Operations & polish (weeks 10–12)

Week 10: Tooling. Goal: be able to run *`npm run build`* for both apps, deploy, and read Cloudflare logs.

- Read `package.json` at the repo root. Run `npm run build:platform` and `npm run build:storefront` locally and watch the output.

- Cloudflare dashboard tour: Workers, D1, R2, Pages, Logs. Find the `saas-platform` worker and its logs.

Week 11: Debugging confidence. Goal: open DevTools on any page and find the cause of "something broke" in under five minutes.

- Re-read Section 7 of this guide. Practice on a deliberately broken local page.
- Read `backend/utils/cdn-cache.js` so you understand why "I just edited this but the storefront still shows the old thing" is almost always a cache question.

Week 12: One real fix. Goal: ship a tiny end-to-end change and write the PR.

- Pick a small bug or copy-fix from the inbox. Trace it through the codebase. Make the change. Run both builds. Commit. Done.
- Update `replit.md` if your change is architectural.

After 90 days you should be able to:

- read any page in the platform or a storefront and explain what's happening,
- find the right backend handler for any API call,
- triage a "something is broken" report from a merchant in under fifteen minutes,
- safely make an isolated change and ship it.

That's the bar. You won't be a senior engineer in 90 days, and you don't need to be — you need to be **dangerous in a small radius** and grow from there.

10. Quick-reference appendix

10.1 Most important files (in order of "read this first")

1. `replit.md` — long-form architecture log. The single source of truth.
2. `backend/workers/index.js` — backend routing entry point.
3. `backend/workers/site-router.js` — how a storefront URL becomes a rendered page.
4. `backend/utils/site-db.js` — how the backend gets the right shard DB.
5. `backend/utils/auth.js` + `backend/workers/platform/auth-worker.js` — authentication.
6. `frontend/src/platform/src/services/api.js` — every frontend call goes through here.
7. `frontend/src/platform/src/context/AuthContext.jsx` — frontend login state.
8. `frontend/src/storefront/src/context/SiteContext.jsx` — active store config on the storefront.
9. `backend/migrations/0001_initial_schema.sql` — the foundation tables.
10. `DATABASE_ARCHITECTURE.md` — deeper dive on shards and usage tracking.

10.2 Common commands

Goal	Command
Run the local dev server	`npm run dev` (from repo root)
Build the platform SPA	`npm run build:platform`
Build the storefront SPA	`npm run build:storefront`
Build both	`npm run build`
Deploy backend (Cloudflare Worker)	`cd backend && npx wrangler deploy`
Tail backend logs	`cd backend && npx wrangler tail`
Run a one-off SQL on the platform DB	`npx wrangler d1 execute DB --remote --command "SELECT ..."`

> **Heads up.** After **any** change to `frontend/src/...`, run **both** build commands (platform + storefront) before committing — see `replit.md`'s "Build Instructions" section.

10.3 "Where do I look if X is broken?"

Symptom	First place to check	Then
Login broken on `flomerce.com`	`auth-worker.js` (`handleLogin`) and the Network tab on `/api/auth/login` .	The `users` table — does the row have `email_verified = 1` ?
"Sign in with Google" doesn't work	Google Cloud console (authorised origins still match), then `handleGoogleLogin` in `auth-worker.js` .	`GOOGLE_CLIENT_ID` matches frontend and backend.
Forgot-password email doesn't arrive	Brevo dashboard activity log.	`email.js` + `email-worker.js` ; the `password_resets` table for the row.
Storefront shows "Site Disabled"	`site-router.js` plan-active check.	The `sites` row's `subscription_plan` and `subscription_expires_at` ; `subscriptions` table for the latest active row.
Storefront shows blank page	DevTools Console → first red error.	Network tab — any failing `.js` or `/api/site` request? Wrong subdomain?
Payments failing on a storefront	The merchant's Razorpay dashboard.	`payment_transactions` for `razorpay_order_id` ; `processed_webhooks` for the webhook event.
Shipping label not generated	`shipping-worker.js` logs at the time of the request.	Merchant's Shiprocket dashboard; the encrypted token in `sites` .
Image upload fails	DevTools Network → `POST /api/upload` .	`upload-worker.js` ; R2 has the `STORAGE` binding; the `site_media` row inserted?
Image visible in admin but 404 on storefront	The R2 prefix `sites/<siteId>/...` exists.	The `site_media` row — was the file evicted by `cleanupOrphanMedia` ?
Edits don't show on storefront	CDN cache hasn't purged.	`cdn-cache.js` — verify `purgeStorefrontCache` ran (Worker logs); manually purge from Cloudflare dashboard.
Push notifications not firing	`web-push.js` + the per-shard `push_subscriptions` table.	`VAPID_*` keys in env.

Symptom	First place to check	Then
WhatsApp messages not sending	Meta Business Suite for the merchant's account.	`whatsapp.js` — template name and language must match exactly.
Translations missing on landing page	`i18n-worker.js`'s EN_CATALOG fingerprint vs the cached locale's stored fingerprint (admin Translations panel).	Did the **backend** redeploy after the JSON edit? (See `replit.md` "DEPLOY GOTCHA".)
`/api/site` is 404 on a custom domain	`sites.custom_domain` matches and `domain_status='verified'`.	Cloudflare custom-hostname record; `cf_hostname_id` on the `sites` row.
Plan limit reached / feature locked	`usage-tracker.js` — `checkFeatureAccess` / `checkCountLimit`.	The plan tier the merchant is actually on (`subscription_plan`); their `subscriptions` row.

10.4 Final words

You will not remember everything in this guide. You don't need to. **Bookmark the table of contents, keep `replit.md` open in another tab, and lean on the "Where do I look if X is broken?" table when you triage your first incident.** The codebase is small, opinionated, and self-similar — once one feature clicks, the next ten click much faster.

Welcome to the team.